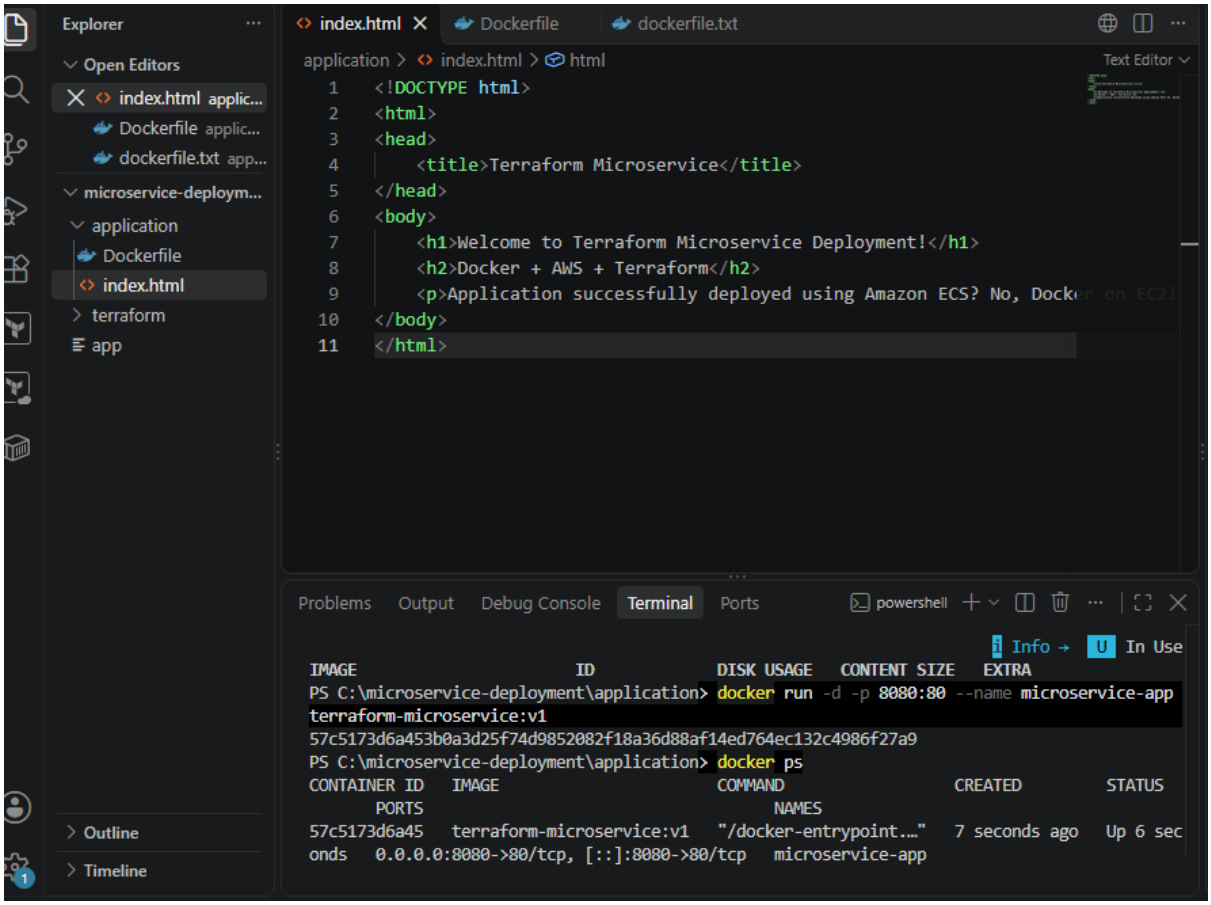
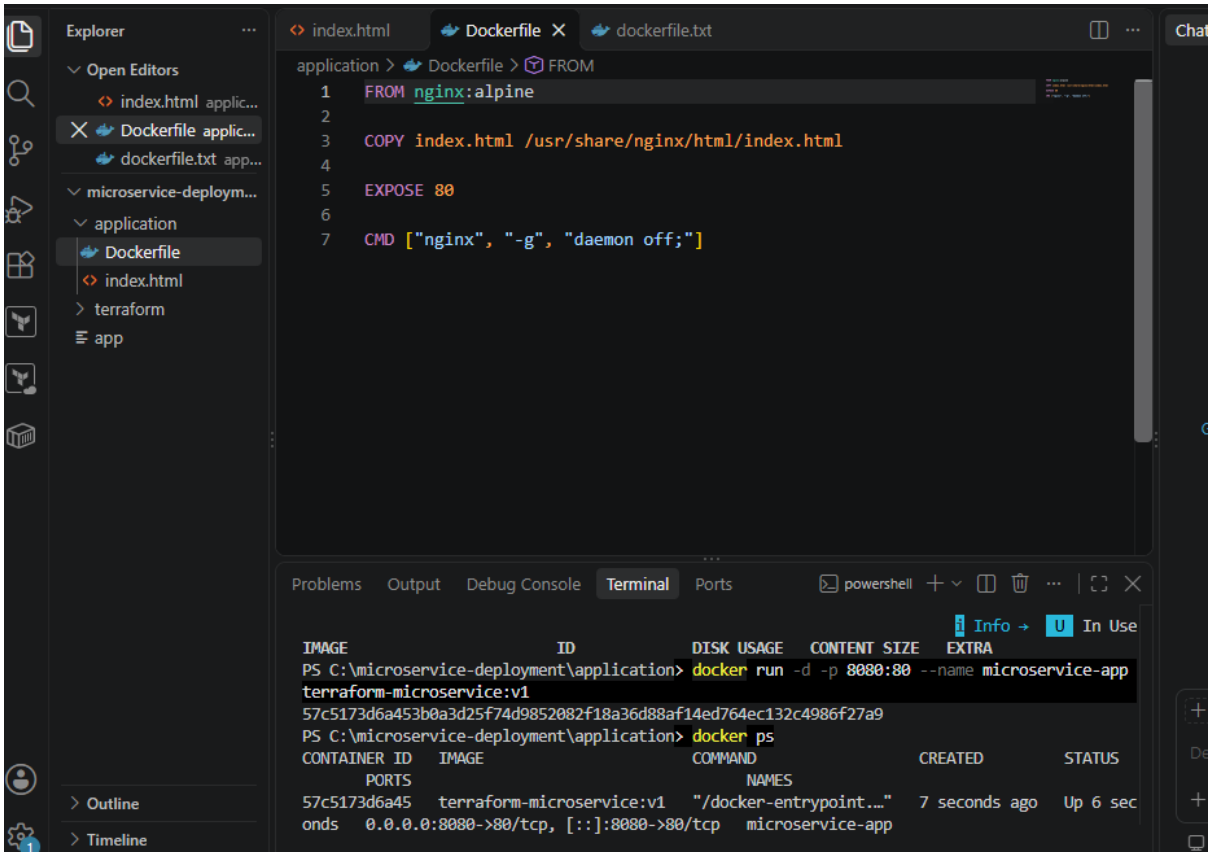
APPLICATION SETUP USING DOCKER

- **Created a project directory for the Terraform microservice deployment.**
- **Created a Dockerfile using the Nginx Alpine base image.**
- **Created a static HTML webpage (`index.html`).**
- **Configured Nginx to serve the custom HTML page.**
- **Exposed port 80 in the Dockerfile.**
- **Built a custom Docker image using Docker.**
- **Ran the Docker container and mapped port 8081 to port 80.**
- **Verified the application locally through a web browser.**

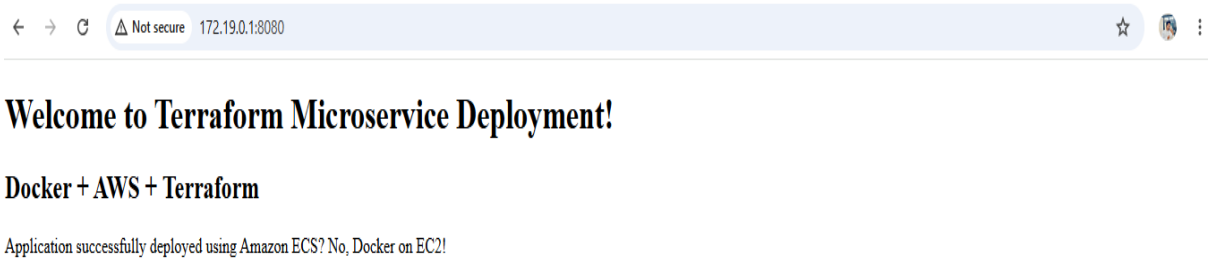
Dockerfile and HTML file.



Dockerfile and running docker container



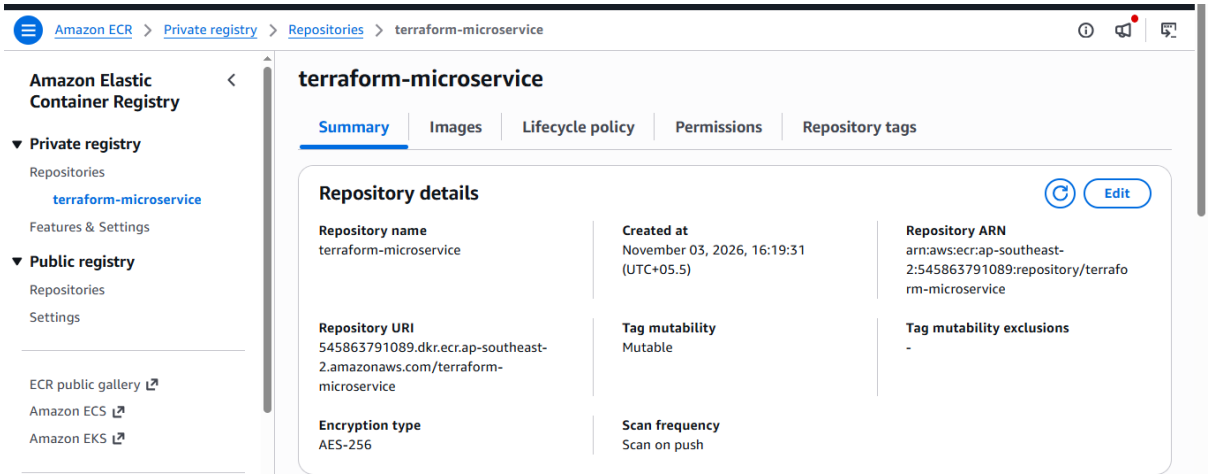
Application webpage in browser.



Created Amazon ECR Repository

Objective: Create a private Docker image repository in AWS Elastic Container Registry (ECR) to store our custom microservice image.

- Configured the AWS provider using Terraform.
- Created a private Amazon ECR repository.
- Configured repository image tag mutability.
- Enabled image scanning on push.
- Resolved IAM permission issues by attaching ECR permissions.
- Successfully provisioned the ECR repository using Terraform.
- Verified the repository in AWS Console.




```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
```

```
}

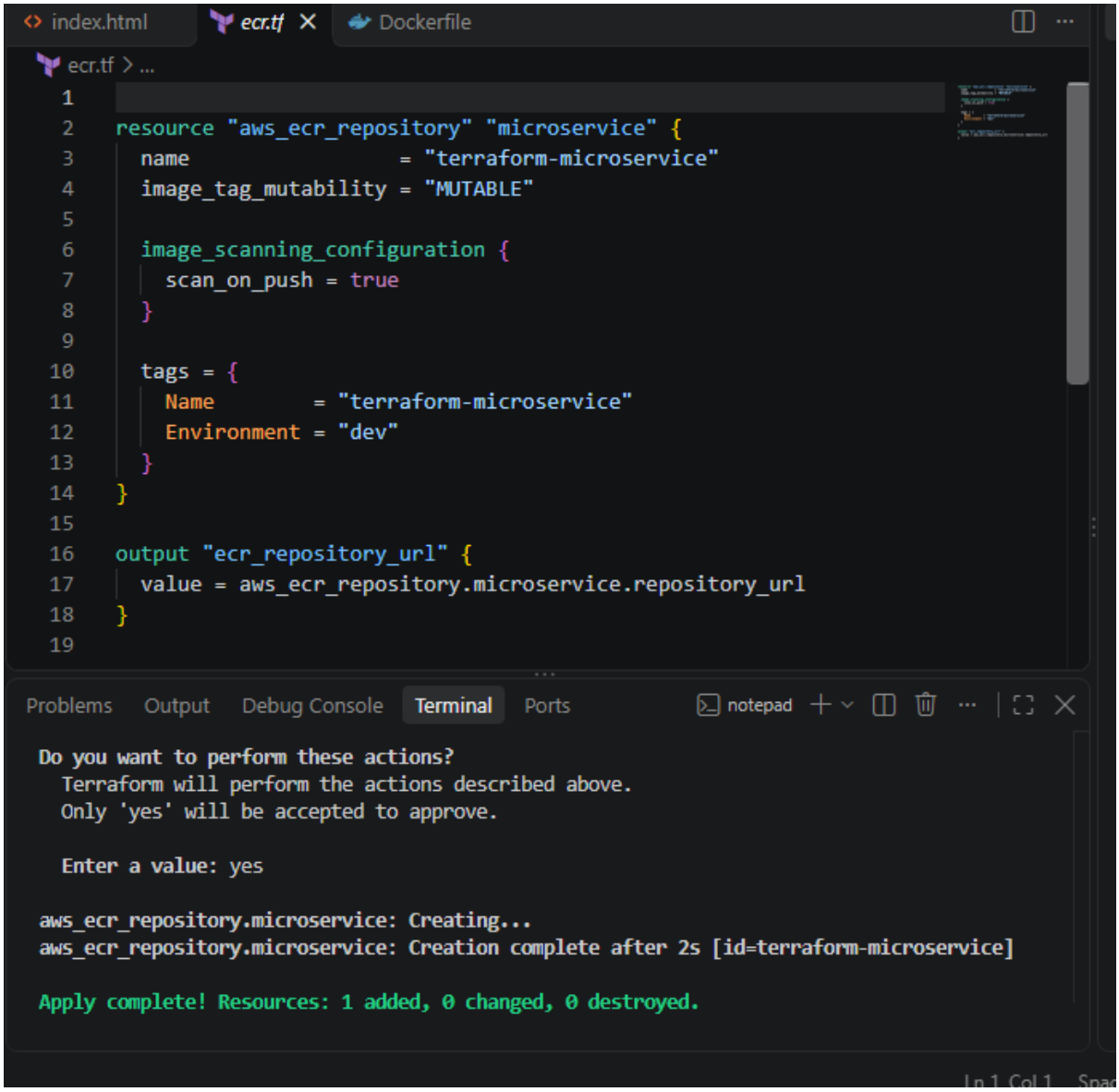
provider "aws" {
  region = "ap-southeast-2"
}

resource "aws_ecr_repository" "microservice" {
  name                 = "terraform-microservice"
  image_tag_mutability = "MUTABLE"

  image_scanning_configuration {
    scan_on_push = true
  }

  tags = {
    Name        = "terraform-microservice"
    Environment = "dev"
  }
}

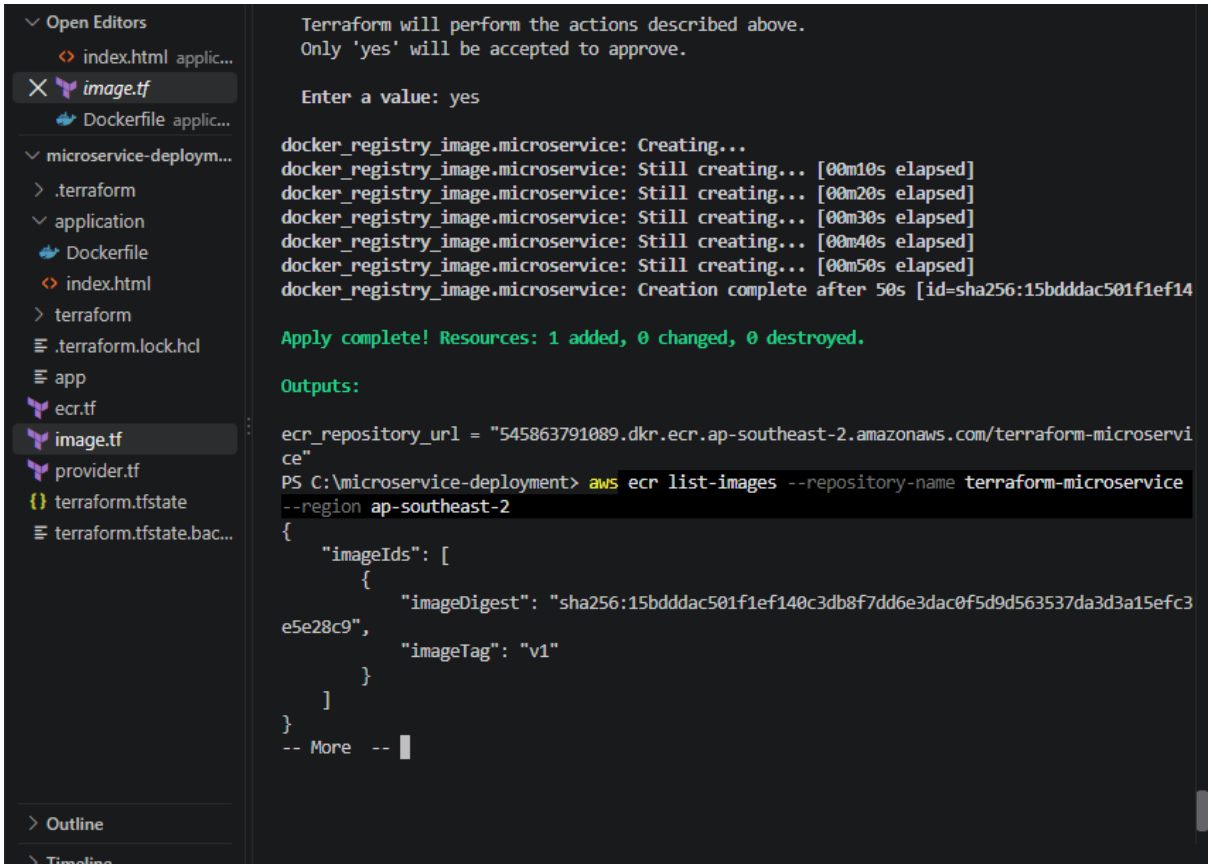
output "ecr_repository_url" {
  value = aws_ecr_repository.microservice.repository_url
}
```



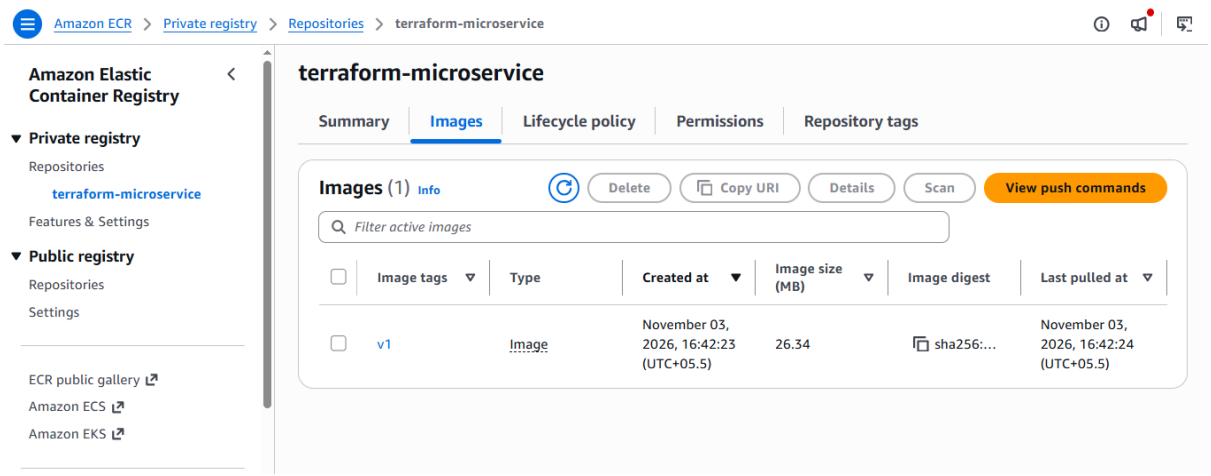
Pushed Docker Image to Amazon ECR using Terraform

- Configured the Terraform Docker provider.
- Added the required AWS ECR authentication data source.
- Tagged the local Docker image with the ECR repository URL.
- Configured Terraform to push the Docker image to Amazon ECR.
- Resolved duplicate resource and provider configuration errors.

- Initialized and validated the Terraform configuration.
- Pushed the Docker image to the ECR repository with the `v1` tag.



Verified the uploaded image in AWS ECR.



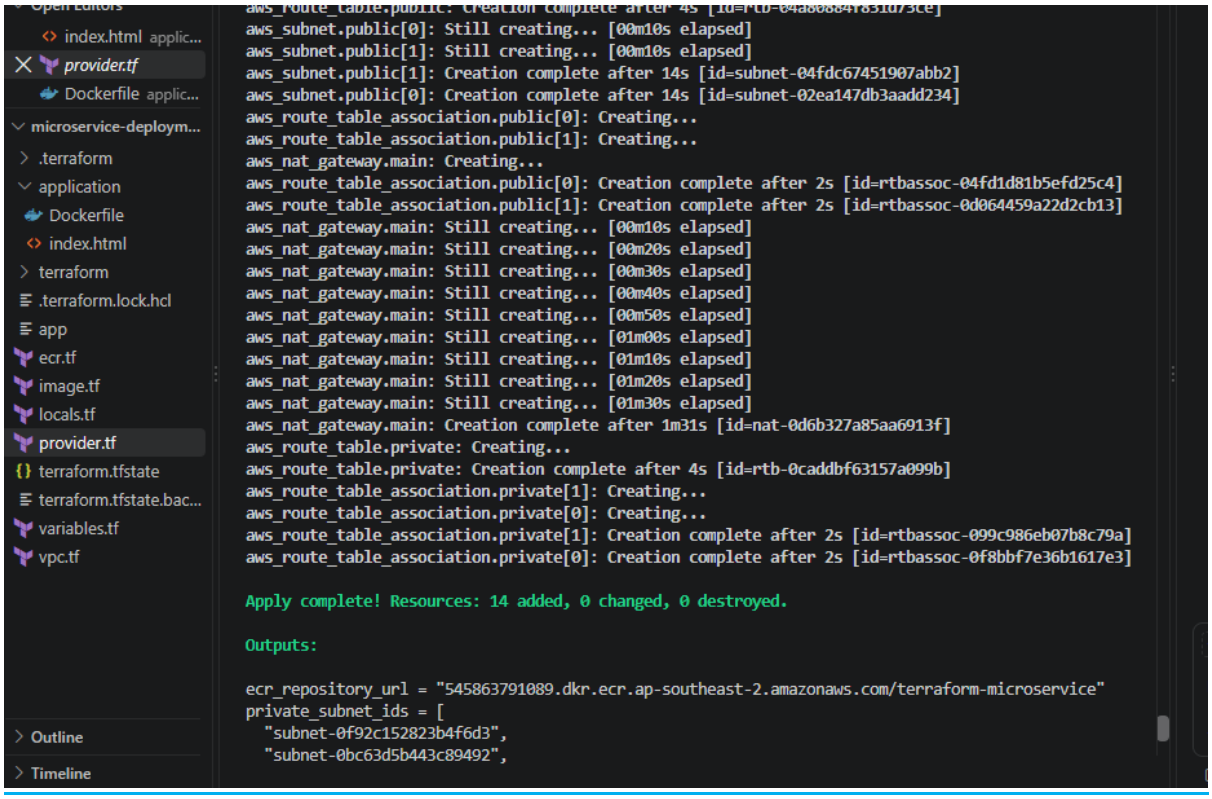
Created VPC and Networking Infrastructure using Terraform

AWS Networking

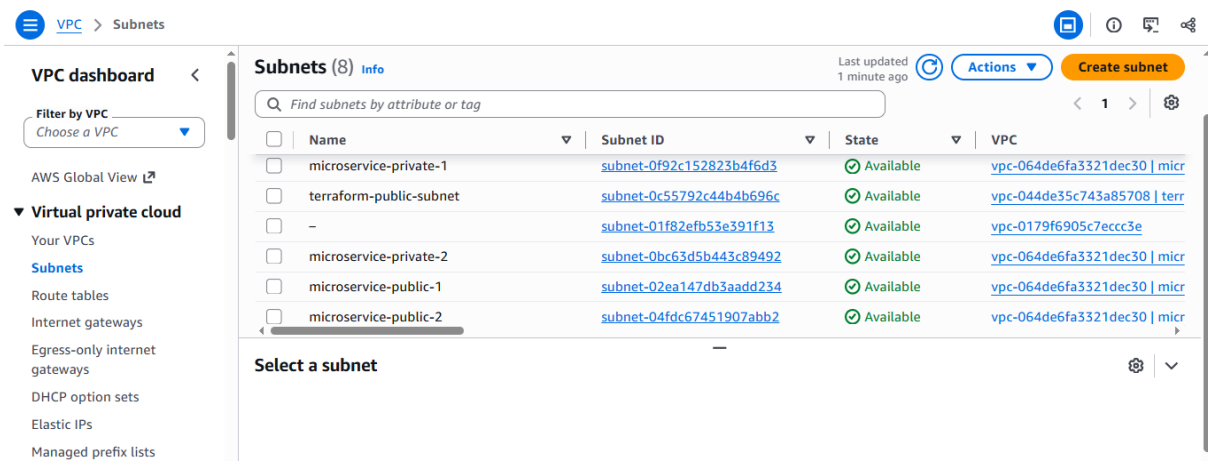
Objective: Create the networking infrastructure required to deploy our microservice securely.

- One custom VPC.
- Two public subnets for the Application Load Balancer.
- Two private subnets for application deployment.
- One Internet Gateway.
- One NAT Gateway for outbound internet access from the private subnet.
- Public and private route tables.
- Security groups for ALB, EC2 and SSH access.

- Created a custom AWS VPC using Terraform.
- Configured VPC CIDR block `10.0.0.0/16`.
- Created two public subnets across two Availability Zones.
- Created two private subnets across two Availability Zones.
- Created and attached an Internet Gateway.
- Configured public and private route tables.
- Created subnet route table associations.
- Allocated an Elastic IP for the NAT Gateway.
- Created a NAT Gateway for private subnet outbound internet connectivity.
- Configured DNS support and DNS hostnames.
- Verified networking resources using Terraform.


index.html applic...
provider.tf
Dockerfile applic...
microservice-deploym...
.terraform
application
Dockerfile
index.html
terraform
.terraform.lock.hcl
app
ecr.tf
image.tf
locals.tf
provider.tf
terraform.tfstate
terraform.tfstate.bac...
variables.tf
vpc.tf
Outline
Timeline
aws_subnet.public[0]: Still creating... [00m10s elapsed]
aws_subnet.public[1]: Still creating... [00m10s elapsed]
aws_subnet.public[1]: Creation complete after 14s [id=subnet-04fdc67451907abb2]
aws_subnet.public[0]: Creation complete after 14s [id=subnet-02ea147db3aadd234]
aws_route_table_association.public[0]: Creating...
aws_route_table_association.public[1]: Creating...
aws_nat_gateway.main: Creating...
aws_route_table_association.public[0]: Creation complete after 2s [id=rtbassoc-04fd1d81b5efd25c4]
aws_route_table_association.public[1]: Creation complete after 2s [id=rtbassoc-0d064459a22d2cb13]
aws_nat_gateway.main: Still creating... [00m10s elapsed]
aws_nat_gateway.main: Still creating... [00m20s elapsed]
aws_nat_gateway.main: Still creating... [00m30s elapsed]
aws_nat_gateway.main: Still creating... [00m40s elapsed]
aws_nat_gateway.main: Still creating... [00m50s elapsed]
aws_nat_gateway.main: Still creating... [01m00s elapsed]
aws_nat_gateway.main: Still creating... [01m10s elapsed]
aws_nat_gateway.main: Still creating... [01m20s elapsed]
aws_nat_gateway.main: Still creating... [01m30s elapsed]
aws_nat_gateway.main: Creation complete after 1m31s [id=nat-0d6b327a85aa6913f]
aws_route_table.private: Creating...
aws_route_table.private: Creation complete after 4s [id=rtb-0caddbf63157a099b]
aws_route_table_association.private[1]: Creating...
aws_route_table_association.private[0]: Creating...
aws_route_table_association.private[1]: Creation complete after 2s [id=rtbassoc-099c986eb07b8c79a]
aws_route_table_association.private[0]: Creation complete after 2s [id=rtbassoc-0f8bbf7e36b1617e3]
Apply complete! Resources: 14 added, 0 changed, 0 destroyed.
Outputs:
ecr_repository_url = "545863791089.dkr.ecr.ap-southeast-2.amazonaws.com/terraform-microservice"
private_subnet_ids = [
"subnet-0f92c152823b4f6d3",
"subnet-0bc63d5b443c89492",

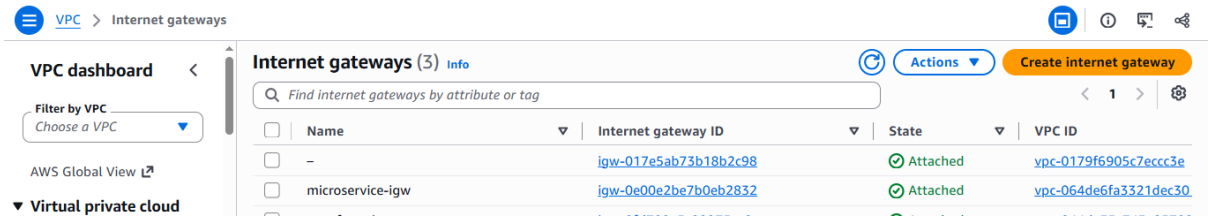
Subnets


VPC > Subnets
VPC dashboard
Filter by VPC
Choose a VPC
AWS Global View
Virtual private cloud
Your VPCs
Subnets
Route tables
Internet gateways
Egress-only internet gateways
DHCP option sets
Elastic IPs
Managed prefix lists
Subnets (8) Info
Last updated 1 minute ago
Actions
Create subnet
Find subnets by attribute or tag
Name
Subnet ID
State
VPC
microservice-private-1 subnet-0f92c152823b4f6d3 Available vpc-064de6fa3321dec30 | micr
terraform-public-subnet subnet-0c55792c44b4b696c Available vpc-044de35c743a85708 | terr
– subnet-01f82efb53e391f13 Available vpc-0179f6905c7eccc3e
microservice-private-2 subnet-0bc63d5b443c89492 Available vpc-064de6fa3321dec30 | micr
microservice-public-1 subnet-02ea147db3aadd234 Available vpc-064de6fa3321dec30 | micr
microservice-public-2 subnet-04fdc67451907abb2 Available vpc-064de6fa3321dec30 | micr
Select a subnet

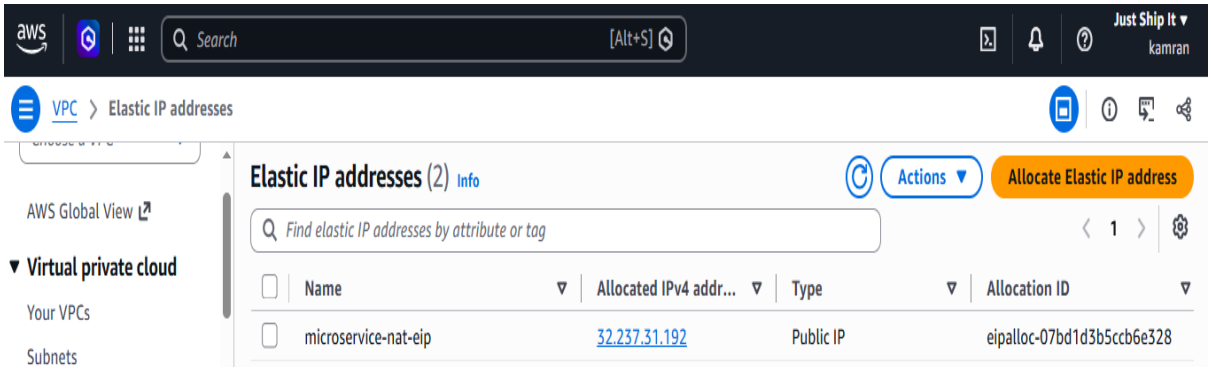
Route tables



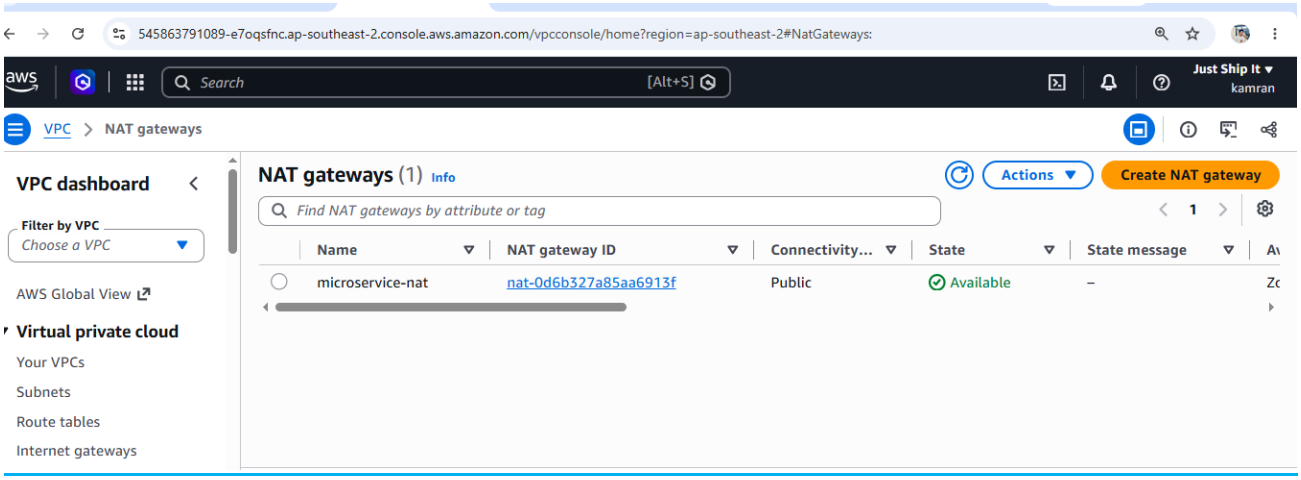
Internet gatewaY



Elastic ip



Nat gateway



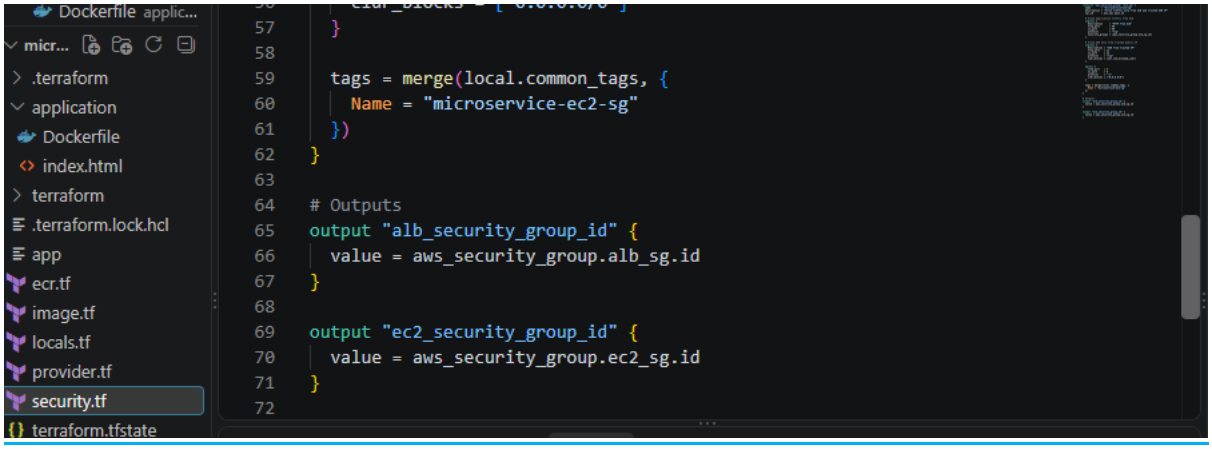
Create Security Groups using Terraform

AWS Security

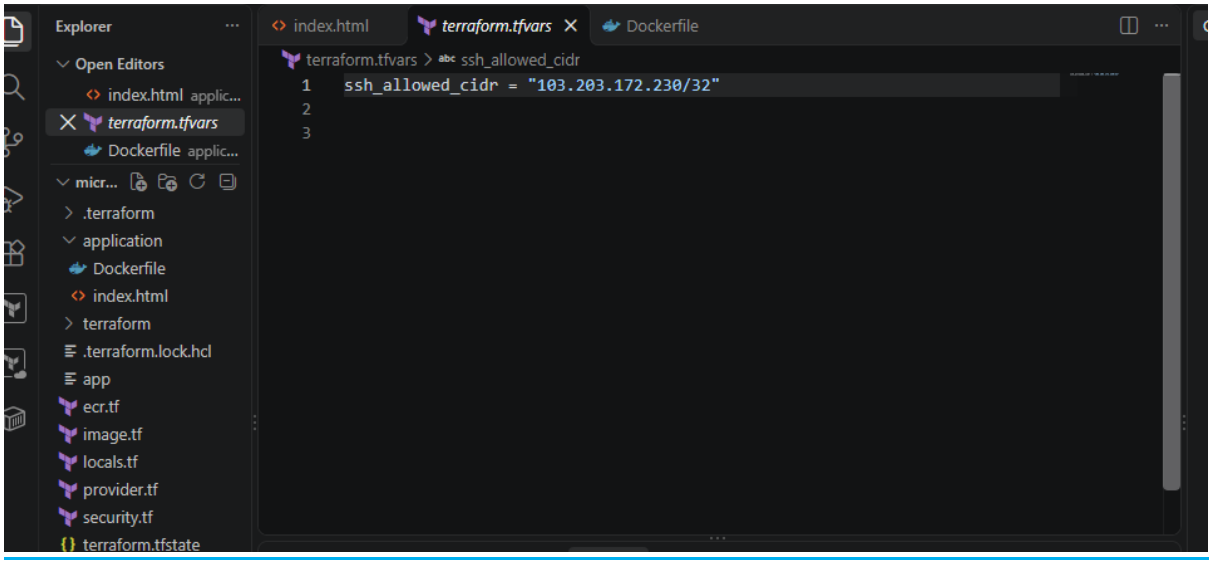
Objective: Configure security groups to control traffic between the internet, Application Load Balancer and EC2 instance.

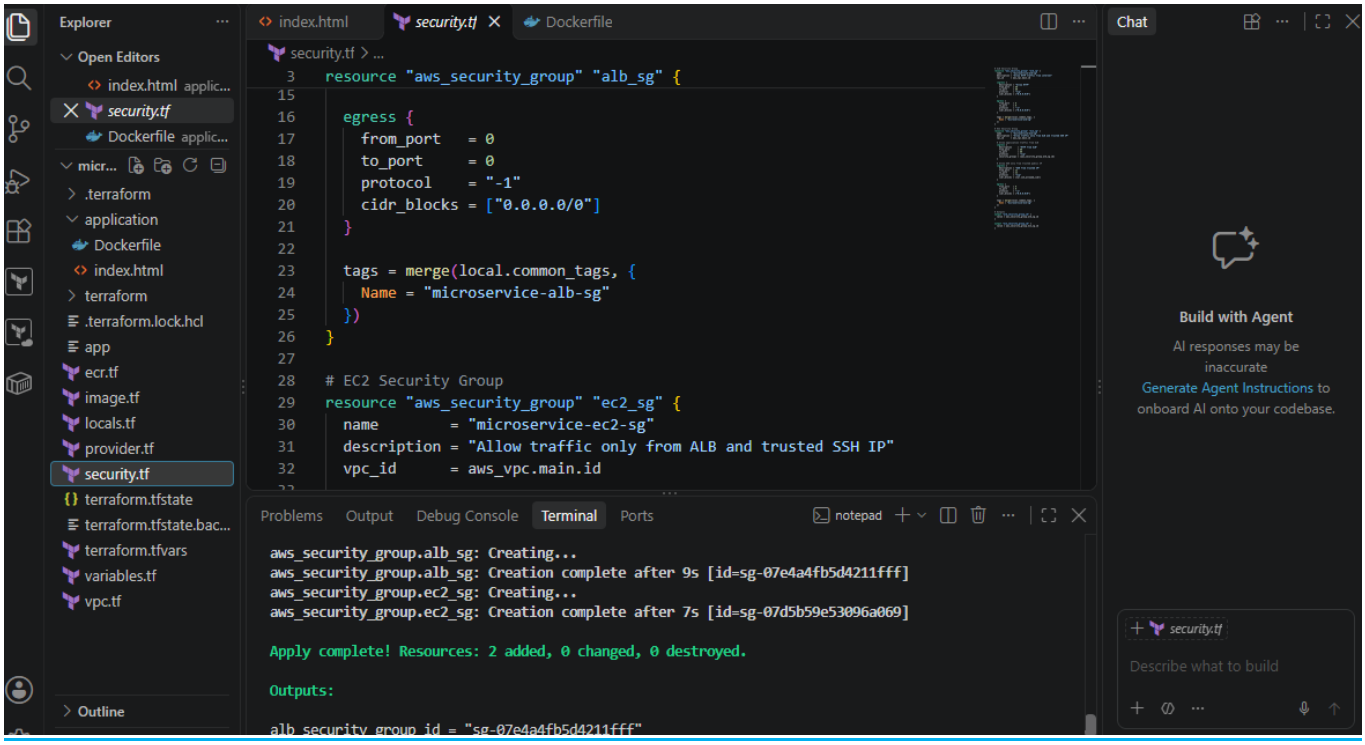
Security Groups Configuration

- Created an ALB security group using Terraform.
- Allowed HTTP traffic on port 80 from the internet to the ALB.
- Created an EC2 security group.
- Allowed application traffic only from the ALB security group.
- Restricted SSH access to a specific public IP address.
- Configured outbound traffic rules.
- Applied security groups to the custom VPC.
- Verified security group rules in AWS Console.


Dockerfile applic...
micr...
.terraform
application
Dockerfile
index.html
terraform
.terraform.lock.hcl
app
ecr.tf
image.tf
locals.tf
provider.tf
security.tf
terraform.tfstate
57 }
58
59 tags = merge(local.common_tags, {
60 Name = "microservice-ec2-sg"
61 })
62 }
63
64 # Outputs
65 output "alb_security_group_id" {
66 value = aws_security_group.alb_sg.id
67 }
68
69 output "ec2_security_group_id" {
70 value = aws_security_group.ec2_sg.id
71 }
72

Tf.vars file

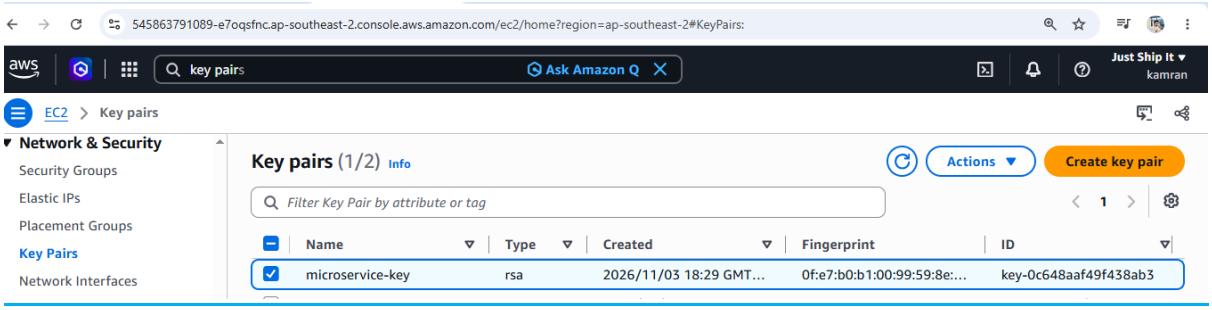

Explorer
Open Editors
index.html applic...
terraform.tfvars
Dockerfile applic...
micr...
.terraform
application
Dockerfile
index.html
terraform
.terraform.lock.hcl
app
ecr.tf
image.tf
locals.tf
provider.tf
security.tf
terraform.tfstate
index.html
terraform.tfvars
Dockerfile
terraform.tfvars > ssh_allowed_cidr
1 ssh_allowed_cidr = "103.203.172.230/32"
2
3



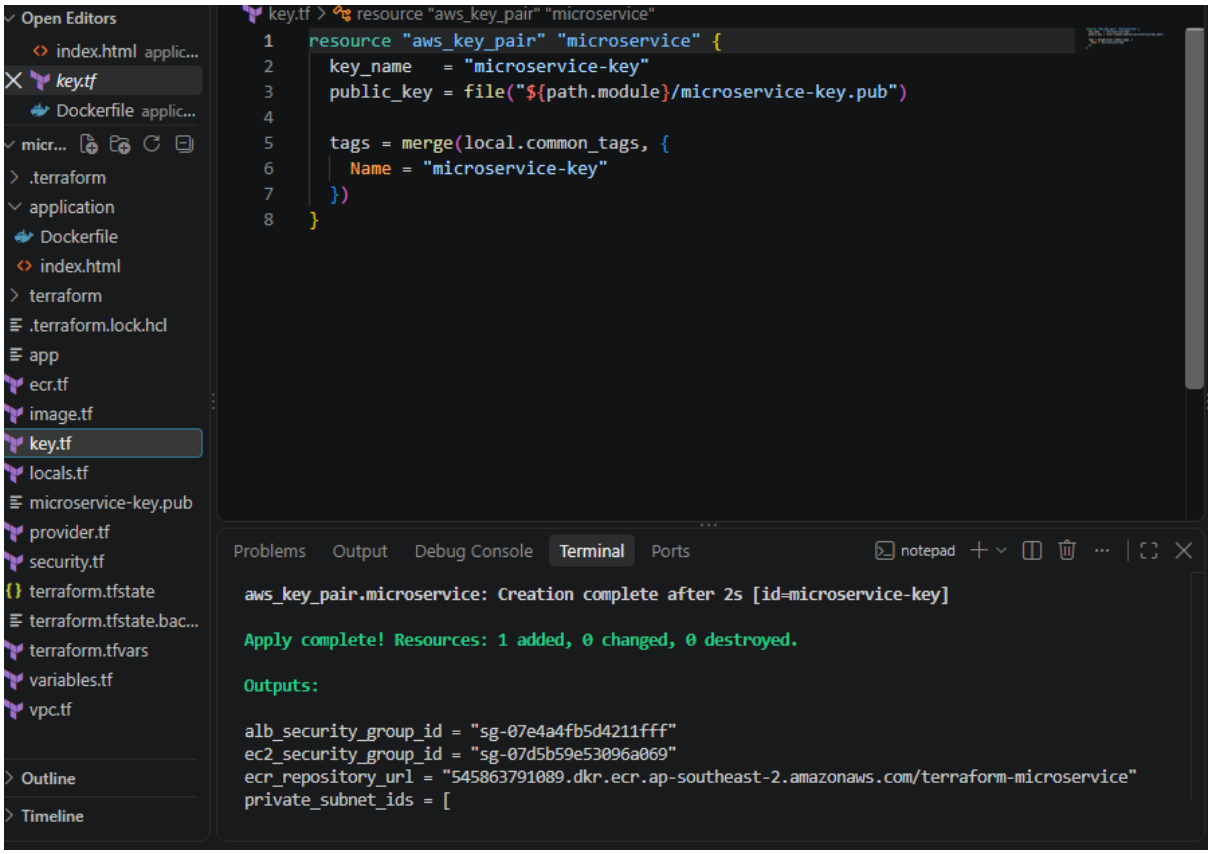
Created SSH Key Pair and Register in AWS

- Generated an RSA SSH key pair using PowerShell.
- Created a public key and private key.
- Stored the private key securely on the local machine.
- Copied the public key into the Terraform project directory.
- Created `key.tf` to manage the AWS key pair.
- Used Terraform to register the public key in AWS.
- Verified the `microservice-key` in AWS EC2 → Key Pairs.
- Successfully completed Terraform validation and deployment.

Key Pair

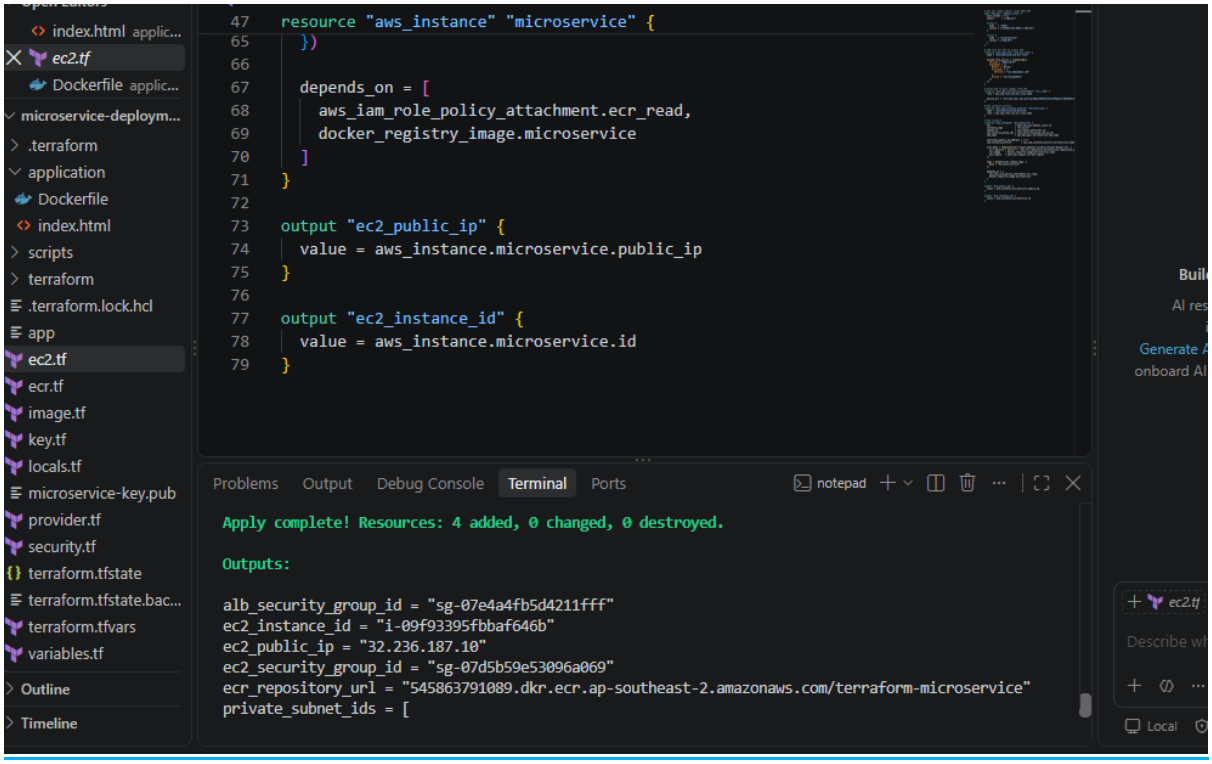


Key.tf

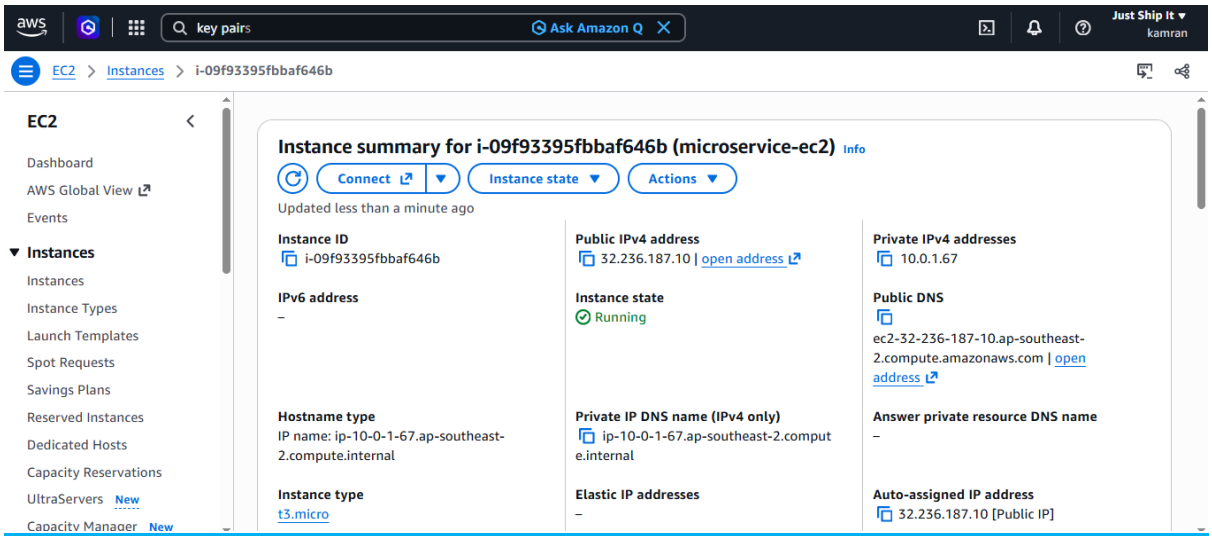


Provisioned EC2 Instance and Install Docker

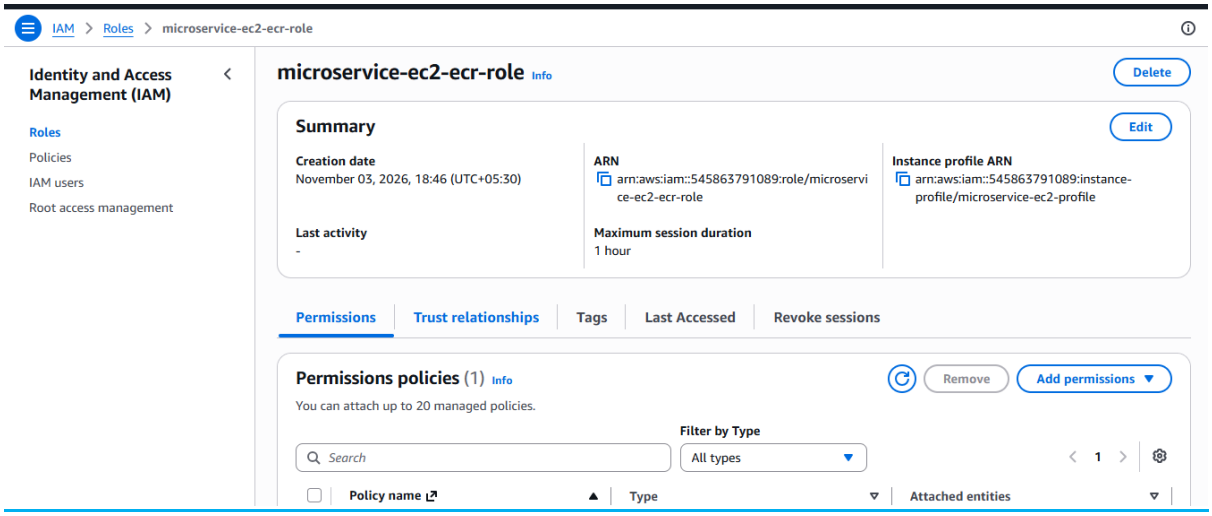
- Created an EC2 instance using Terraform.
- Selected Amazon Linux 2023 AMI.
- Configured EC2 instance type as `t3.micro`.
- Attached the existing SSH key pair to the EC2 instance.
- Created an IAM role for EC2 to access Amazon ECR.
- Attached the `AmazonEC2ContainerRegistryReadOnly` policy.
- Created an IAM instance profile.
- Configured the Docker installation script using EC2 user data.
- Installed and started Docker automatically on EC2.
- Authenticated EC2 with Amazon ECR.
- Pulled the Docker image from ECR.
- Deployed the microservice container on port 80.
- Verified the EC2 instance and running Docker container.



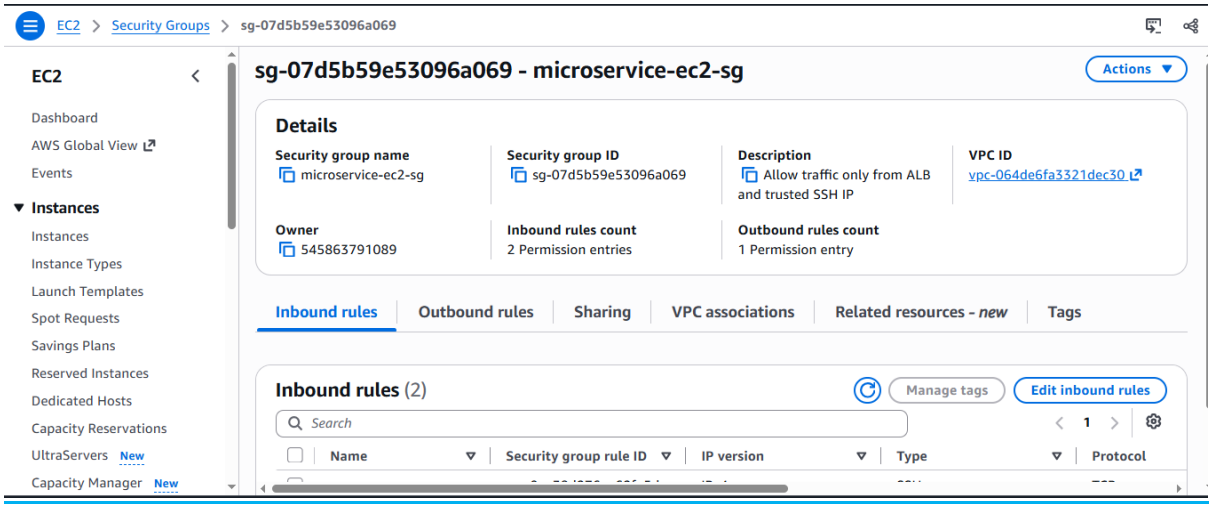
Created an EC2 instance using Terraform.



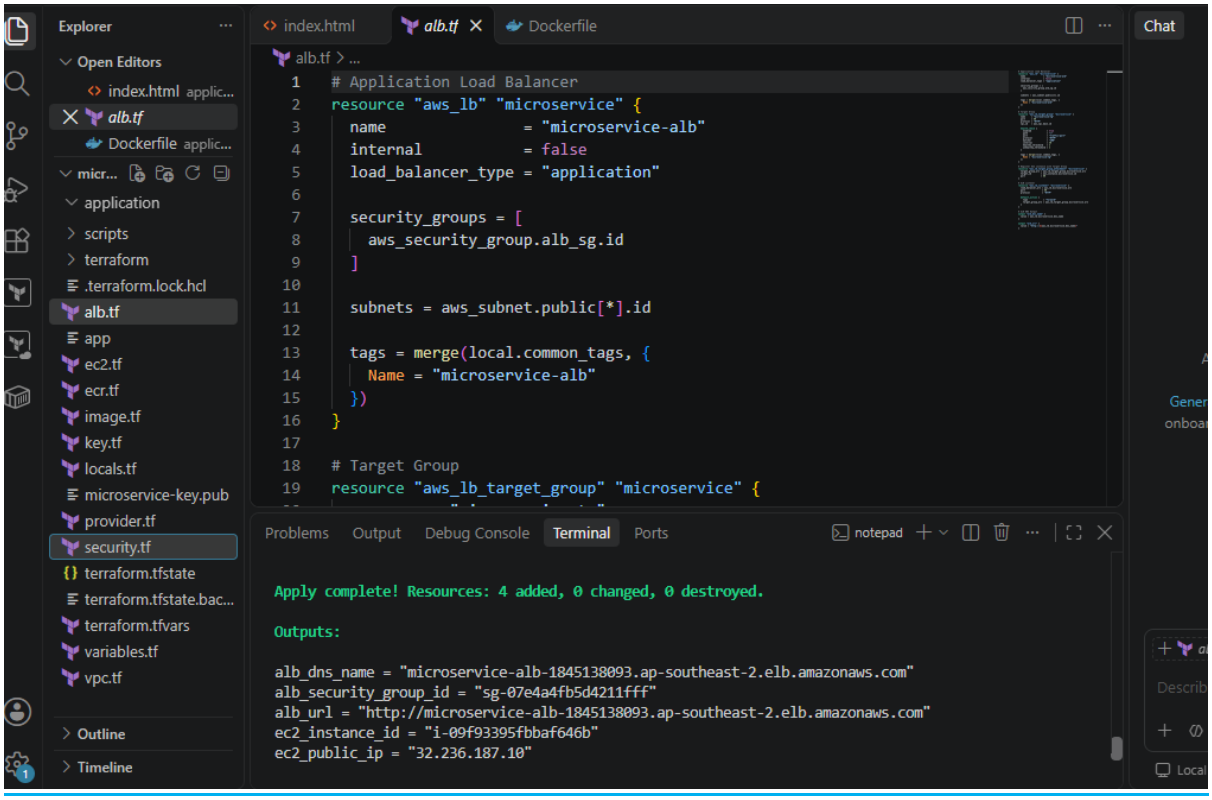
Created an IAM role for EC2 to access Amazon ECR.

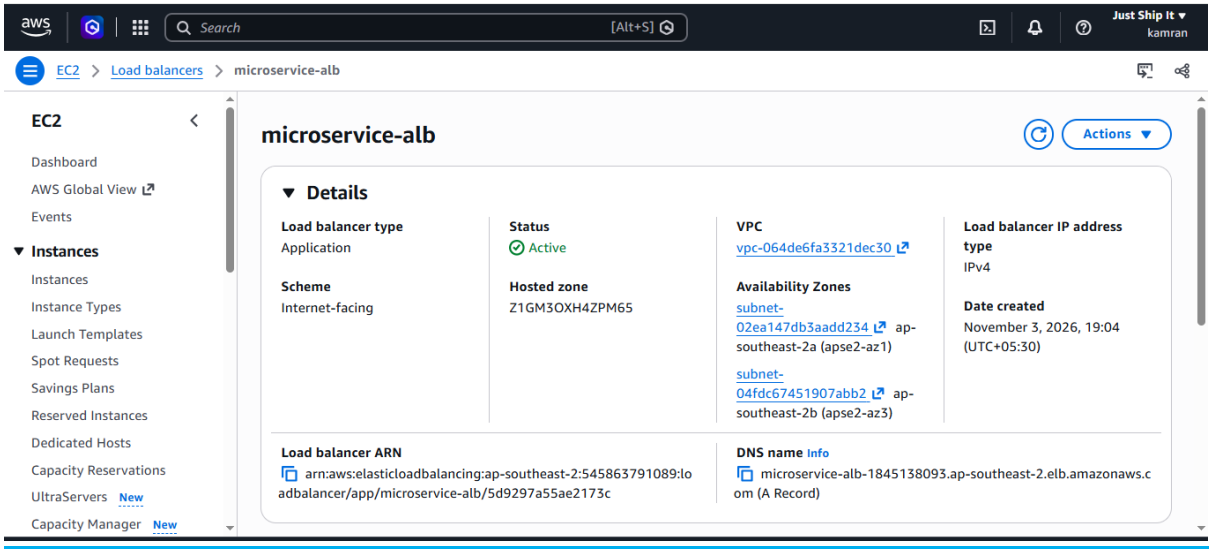


EC2 Security Group showing SSH



Created Application Load Balancer (ALB)



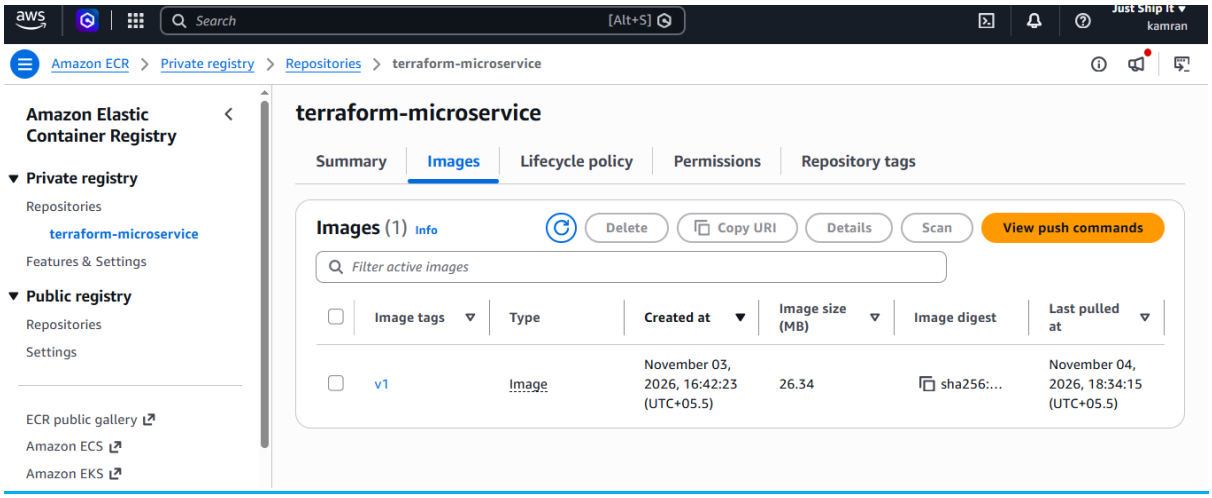


Final Testing and Deployment Verification

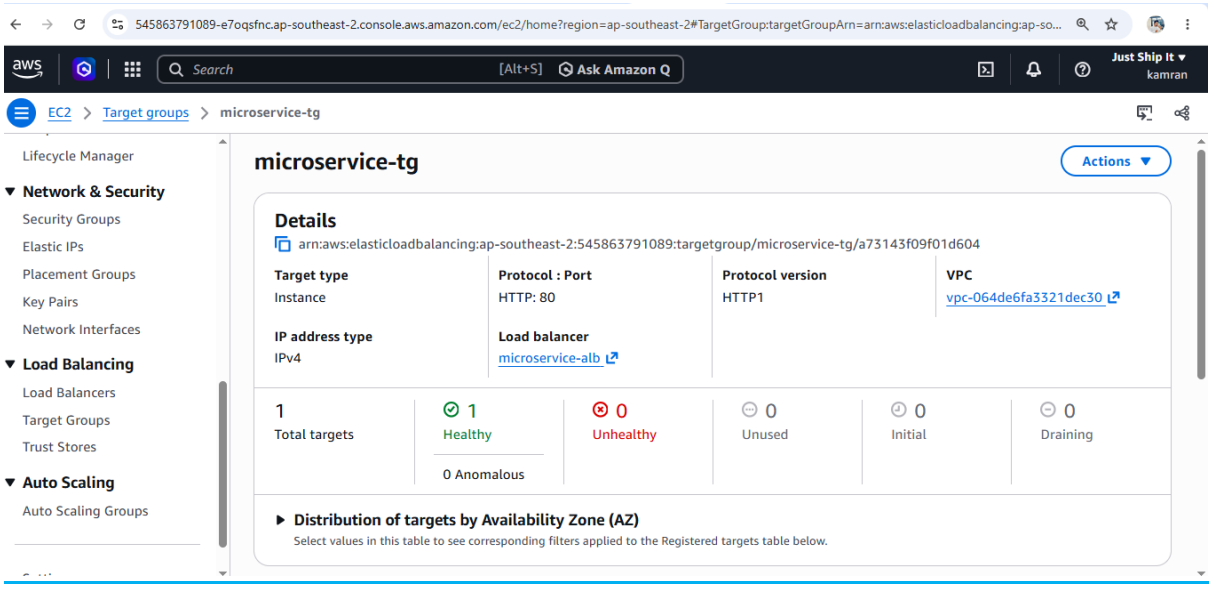
- Created an internet-facing Application Load Balancer.
- Configured an HTTP listener on port 80.
- Created a target group for the EC2 instance.
- Registered the EC2 instance with the target group.
- Configured ALB and EC2 security groups.
- Deployed the Nginx Docker container using an image stored in Amazon ECR.
- Verified the Nginx application returned HTTP `200 OK`.
- Accessed the application successfully through the ALB DNS URL.

```
Terraform has compared your real infrastructure against your configuration and found no
differences, so no changes are needed.
PS C:\microservice-deployment> terraform output
alb_dns_name = "microservice-alb-1845138093.ap-southeast-2.elb.amazonaws.com"
alb_security_group_id = "sg-07e4a4fb5d4211fff"
alb_url = "http://microservice-alb-1845138093.ap-southeast-2.elb.amazonaws.com"
ec2_instance_id = "i-09f93395fbbaf646b"
ec2_public_ip = "32.236.187.10"
ec2_security_group_id = "sg-07d5b59e53096a069"
ecr_repository_url = "545863791089.dkr.ecr.ap-southeast-2.amazonaws.com/terraform-microservice"
private_subnet_ids = [
  "subnet-0f92c152823b4f6d3",
  "subnet-0bc63d5b443c89492",
]
public_subnet_ids = [
  "subnet-02ea147db3aadd234",
  "subnet-04fdc67451907abb2",
]
vpc_id = "vpc-064de6fa3321dec30"
PS C:\microservice-deployment>
```

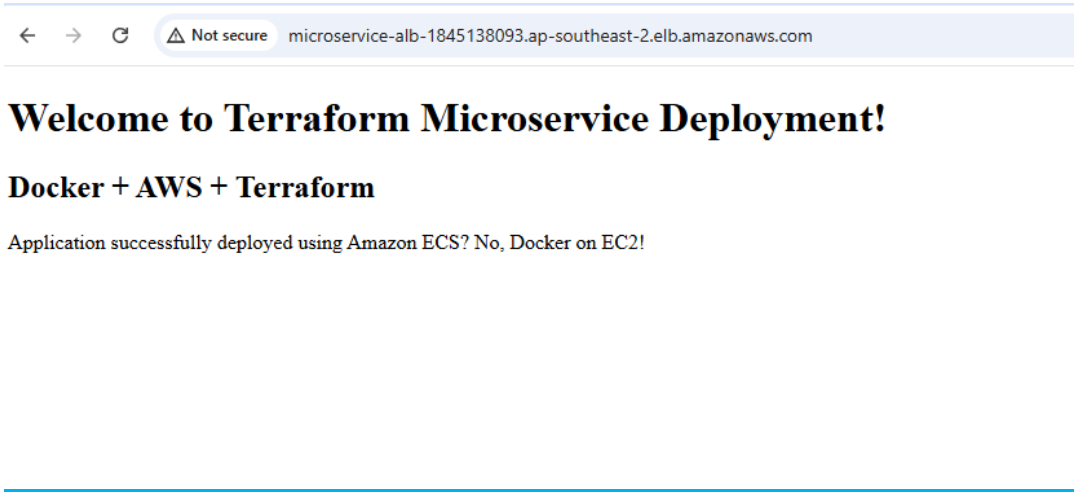
ECR repository showing the v1 image.



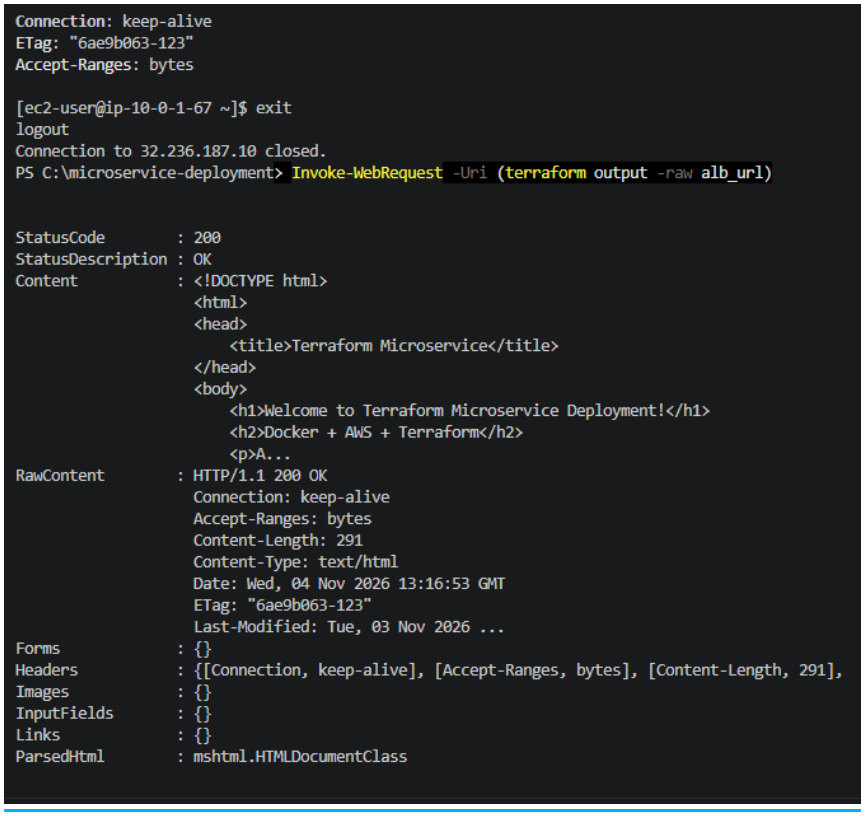
Target group showing Healthy status



Browser displaying application through the ALB URL.



EC2 terminal showing curl -I http://localhost returning 200 ok

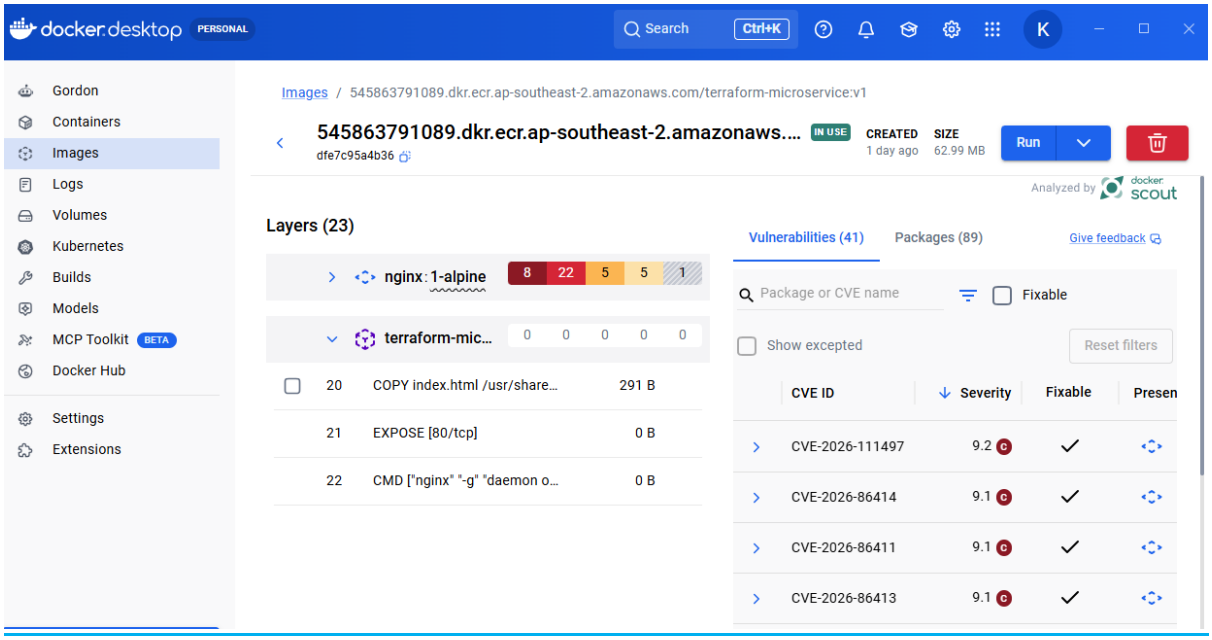


```
Connection: keep-alive
ETag: "6ae9b063-123"
Accept-Ranges: bytes

[ec2-user@ip-10-0-1-67 ~]$ exit
logout
Connection to 32.236.187.10 closed.
PS C:\microservice-deployment> Invoke-WebRequest -Uri (terraform output -raw alb_url)


StatusCode        : 200
StatusDescription : OK
Content           : <!DOCTYPE html>
                    <html>
                    <head>
                        <title>Terraform Microservice</title>
                    </head>
                    <body>
                        <h1>Welcome to Terraform Microservice Deployment!</h1>
                        <h2>Docker + AWS + Terraform</h2>
                        <p>A...
RawContent        : HTTP/1.1 200 OK
                    Connection: keep-alive
                    Accept-Ranges: bytes
                    Content-Length: 291
                    Content-Type: text/html
                    Date: Wed, 04 Nov 2026 13:16:53 GMT
                    ETag: "6ae9b063-123"
                    Last-Modified: Tue, 03 Nov 2026 ...
Forms             : {}
Headers           : {[Connection, keep-alive], [Accept-Ranges, bytes], [Content-Length, 291],
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
```

Docker desktop showing alb.



Running container in docker desktop

